# Assessment 3 Assignment - Reflection.

yogita kumari
Roll no 230101091

Github : https://github.com/yogitaranwa/CS4032A3

When I started this assignment, I had only a theoretical idea of how ML models are actually deployed. Building a natural-language command parser for Kubernetes end to end taught me how many pieces have to fit together before a model becomes a real service: data labelling, training, packaging, CI/CD, orchestration and load testing. The measurements I quote below are stored in the results/ folder and come from the 30 October run of the code in a separate rehearsal repository; I say so because a number is only as honestly as its origin.

The first thing I learnt was that data quality decides everything. Labelling 3,000 commands by hand was impossible, so I used a teacher model and split the work into 15 shards of 200 that could run in parallel and be re-run safely. Cleaning taught me a harder lesson: of the 3000 records, 152 were rejected because the teacher paraphrased a slot instead of copying the command's words, 44 were duplicate and 2,804 were kept. The honest fix was to drop those records and as an improvement, to have the teacher repair only the rejected ones. I learnt to treat the dataset as something engineered and to measure how much data actually survives cleaning.

The second lesson was that a model is only as trustworthy as the gate that let it through. The pipeline enforces linting, tests

an ACL check and a quality gate requiring at least 95% valid JSON. The stored score show why: the untrained base model produced 0% valid JSON, while both fine-tuned students reached 99%. On the strict full-frame metric over 100 validation example that difference is within noise, so I claim only that the students match their teachers.

Packaging and deployment were lessons in constraints. The Docker image had to stay under 2 GB while carrying a model, tokenizer and web server, and came to 1.22 GB; model artefacts live in a release with checksums so the repository stays light and every model is verifiable. All 50 deployment test requests returned HTTP 200 within 1 CPU and 1 GiB, peaking at 358.7 MiB. Reading the request log taught me something subtler one request, "what time does meeting start", returned valid JSON with no slots, a wrong answer that still passes the schema check. A validity gate checks the shape of a reply, not its truth, so correctness must be measured separately by the scoring script.

The Kubernetes scenarios taught me how systems behave under pressure: the autoscaler scaled from 1 to 2 replicas and back with 615 requests and no failures, 1 of 906 requests failed during a rolling update, and a deleted pod recovered in 7s with no failure of 708 requests. In the standard run two replicas received almost the same number of requests, which made load balancing feel real for the first time. Costing changed how I think too: the 0.53 INR per thousand requests rests on a hypothetical rate card, so it is an assumption, whereas the throughput of 4.16 requests per second is a measurement. A number you did not measure is a claim, not a result.

I would still improve a lot: a multi-node cluster, a larger validation set with several training seeds, a repair pass for the paraphrased slots and re-running every measurement from the final repository. My main observation is that the model itself was the easy

part: the hard, valuable work was everything around it the reproducible pipelines, honest measurement, automated gates and clean deployment. I now understand why industry treats MLOps as a discipline of its own, and I feel far more confident taking a model from a notebook to a running, load-tested service.